

LeetCode Deep Dive

From First Instinct to Optimal Algorithm: Union-Find and Graph DFS in Practice
Week 3 Supplement – TOAE209/TOAH209: Design and Analysis of Algorithms

Foreign Trade University

Outline

- 1 A Framework for Attacking a New Graph Problem
- 2 Problem 1 (Medium): Redundant Connection
- 3 Problem 2 (Hard): Critical Connections in a Network
- 4 Summary

Why walk through LeetCode problems?

- This week's LEETCODE.md list already gives you ten Week 3 problems to *practice on your own*. This deck instead **narrates the thinking process** for two of them, end to end, so you can reuse the process on the other eight.
- We pick one **Medium** (*Redundant Connection*) and one **Hard** (*Critical Connections in a Network*) – both graph problems, both solved optimally with tools from this week's lecture: **Union-Find** and **DFS**.
- Spoiler: the two problems turn out to be **mirror images** of each other – one finds an edge that *is* on a cycle (safe to delete), the other finds every edge that is *not* on any cycle (a bridge – unsafe to delete).

The four-step process we will repeat twice

- 1 **First instinct.** Write down the algorithm that follows most directly from re-reading the problem statement – usually a simulate-it-literally or try-every-possibility approach.
- 2 **Measure why it hurts.** Compute the actual complexity, plug in the problem's constraints, and see the number blow up.
- 3 **Find the structural insight.** Ask: *what property of the problem lets me avoid redoing that work?* This is almost always a graph-theoretic fact (a counting argument, a cycle/tree property, a monotonicity).
- 4 **Implement the optimal algorithm,** trace it by hand on a small example, then look for the *programming-level* (not algorithmic) tricks that make the implementation itself correct and fast.

This mirrors the course

Steps 1–3 are exactly the “upper bound via a clever algorithm” story from this week’s Union-Find lecture ($O(n^2) \rightarrow O(n \log n) \rightarrow O(n \alpha(n))$); step 4 is what turns a correct algorithm into working code.

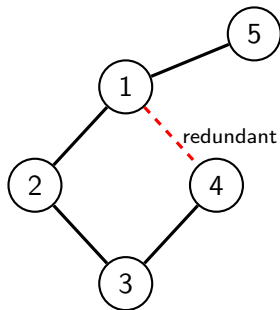
Problem statement

LeetCode 684 – Redundant Connection (Medium)

A tree with n nodes labeled $1, \dots, n$ has exactly $n - 1$ edges. You are given the tree's edge list *plus one extra edge*, so the input has exactly n edges and contains exactly **one cycle**. Return an edge that can be removed so the graph becomes a tree again. If several edges lie on the cycle, return the one that occurs **last** in the input.

- Constraints (LeetCode): $3 \leq n \leq 1000$, exactly n edges, no self-loops, no duplicate edges, input represents a connected graph.
- This is precisely “detect the one cycle in an otherwise-tree graph,” the undirected cycle-detection question from this week’s lecture.

Worked example



`connections = [[1,2],[2,3],[3,4],
[1,4],[1,5]]`

- Edges 1–2–3–4–1 form the unique cycle.
- Edge 1–5 is a harmless pendant, added *after* the cycle already closed.
- **Answer:** [1,4] – the edge that, when scanned left to right, is the one that closes the cycle.

First instinct: try removing each candidate

Most direct reading of the problem

“Return an edge whose removal makes it a tree” $\Rightarrow$ for each edge e (scanning from the *end*, since ties break toward the last edge), remove e and check: is the remaining graph connected with exactly $n - 1$ edges?

```
1: for  $i \leftarrow m - 1$  downto 0 do  
2:    $G' \leftarrow$  all edges except edge  $i$   
3:   if ISCONNECTED( $G'$ ) via BFS/DFS from node 1 then  
4:     return edge  $i$   
5:   end if  
6: end for
```

$\triangleright m = n$ here

Why this is bad

- Each connectivity check is a full BFS/DFS: $O(n + m) = O(n)$ (since $m = n$ here).
- In the worst case (the cycle is the *whole* graph, e.g. a Hamiltonian cycle $1-2-\dots-n-1$), you must try up to n candidate removals before finding the right one.
- Total: $O(n) \times O(n) = O(n^2)$ – and you **rebuild the entire connectivity picture from scratch** on every single trial, throwing away almost all of the previous trial's work.

The smell to notice

Whenever an algorithm says “try removing/adding X , then recheck everything,” ask: *does removing X actually change much, or am I recomputing the same answer n times?* That question is what leads to the insight below.

The structural insight

- The input has *exactly* n edges on n nodes and is connected $\Rightarrow$ it is a tree ($n - 1$ edges) plus **exactly one** extra edge $\Rightarrow$ **exactly one cycle exists, and exactly one edge in the whole scan will ever connect two nodes that are already in the same component.**
- So you never need to *remove and recheck* anything. Just **add edges left to right** and watch for the first moment two endpoints are already connected – that edge is provably the (unique) answer.
- This is exactly the graph-traversal insight from this week's lecture: “a new edge closes a cycle iff its endpoints are already in the same connected component.” Union-Find answers *that specific question* in near-constant time.

Optimal algorithm: single Union-Find pass

```
1: function FIND( $x$ )
2:   while  $parent[x] \neq x$  do
3:      $parent[x] \leftarrow parent[parent[x]]$ 
4:      $x \leftarrow parent[x]$ 
5:   end while
6:   return  $x$ 
7: end function
8: function REDUNDANTCONNECTION( $edges$ )
9:   initialize  $parent[i] \leftarrow i$  for all nodes
10:  for  $(u, v)$  in  $edges$  do
11:     $r_u, r_v \leftarrow FIND(u), FIND(v)$ 
12:    if  $r_u = r_v$  then
13:      return  $(u, v)$ 
14:    end if
15:     $parent[r_v] \leftarrow r_u$ 
16:  end for
17: end function
```

▷ path halving

▷ scan left to right

▷ first edge closing the (unique) cycle

▷ union

Cost: $O(n\alpha(n))$ with union by rank + path compression – this week's bound.

Trace on the worked example

connections = $[[1,2], [2,3], [3,4], [1,4], [1,5]]$

edge	find(u)	find(v)	same root?	action	components after
(1,2)	1	2	no	union	$\{1,2\}, \{3\}, \{4\}, \{5\}$
(2,3)	1	3	no	union	$\{1,2,3\}, \{4\}, \{5\}$
(3,4)	1	4	no	union	$\{1,2,3,4\}, \{5\}$
(1,4)	1	1	yes	stop	—

Answer

Edge (1, 4) is the first (and, by the problem's guarantee, only) edge whose endpoints already share a root. Edge (1, 5) is never even reached.

Complexity: naive vs. optimal

Approach	Time	Idea
Remove-and-recheck (naive)	$O(n^2)$	re-run BFS/DFS after each trial removal
Rebuild via BFS/DFS components	$O(n^2)$	re-run BFS/DFS after each new edge
Union-Find, single pass	$O(n \alpha(n))$	incremental, online connectivity test

- Note that plain BFS/DFS recomputation *per edge* is also $O(n^2)$: it is the *online* nature of the query (“are u, v connected *right now*, after only some edges have been added?”) that Union-Find is built for – not just raw graph size.

Python implementation (part 1: the Union-Find class)

```
1 class UnionFind:
2     def __init__(self, n):
3         self.parent = list(range(n + 1))    # 1-indexed nodes
4         self.rank = [0] * (n + 1)
5
6     def find(self, x):
7         while self.parent[x] != x:
8             self.parent[x] = self.parent[self.parent[x]]    # path halving
9             x = self.parent[x]
10        return x
11
12    def union(self, a, b):
13        ra, rb = self.find(a), self.find(b)
14        if ra == rb:
15            return False    # already connected: cycle-closing edge
16        if self.rank[ra] < self.rank[rb]:
17            ra, rb = rb, ra
18        self.parent[rb] = ra
19        if self.rank[ra] == self.rank[rb]:
20            self.rank[ra] += 1
21        return True
```

Python implementation (part 2: the driver)

```
1 def findRedundantConnection(edges):  
2     uf = UnionFind(len(edges))  
3     for u, v in edges:  
4         if not uf.union(u, v):  
5             return [u, v]  
6     return [] # unreachable given the problem's guarantee
```

The very first edge that fails to union (because its endpoints are already joined) is returned immediately – no need to look at the rest of the list.

Implementation tips & tricks (the programming side)

These are *not* about the algorithm's Big-O – the algorithm above is already optimal. They are about turning it into code that is actually correct and fast.

- **Iterative find, not recursive.** A recursive `find` walks up the parent chain via function calls. On an adversarial input where edges are added as a long chain (1-2, 2-3, ...) *without* union by rank, the recursion depth equals the chain length – for n near Python's default recursion limit (1000), that is a real `RecursionError`, not just a slowdown. The `while` loop above has no such limit.
- **Path halving vs. full path compression.** `parent[x] = parent[parent[x]]` inside the loop is a cheap one-line trick that still gives the same $O(\alpha(n))$ amortized bound as full recursive compression, with no extra pass over the path.
- **Union by rank matters even for small n .** Without it, a chain input produces a chain-shaped tree ($O(n)$ per `find`); with it, tree height is provably $O(\log n)$ even before compression kicks in.

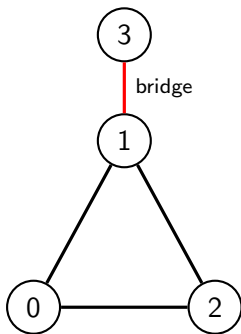
Problem statement

LeetCode 1192 – Critical Connections in a Network (Hard)

There are n servers labeled $0, \dots, n - 1$ connected by an undirected, connected graph with edge list `connections` (no self-loops, no duplicate edges). A **critical connection** (a **bridge**) is an edge whose removal disconnects the network. Return *all* critical connections.

- Constraints (LeetCode): $n \leq 10^5$, up to 10^5 edges.
- This is the exact mirror of Problem 1: instead of finding the one edge that *is* on a cycle, find every edge that is **on no cycle at all**.

Worked example



`connections = [[0,1],[1,2],[2,0],[1,3]]`

- 0,1,2 form a triangle: every edge among them lies on a cycle, so removing any *one* of them still leaves the network connected.
- Edge 1–3 is a pendant, on **no** cycle: removing it strands server 3.
- **Answer:** `[[1,3]]`.

First instinct: remove each edge and recheck

Most direct reading of the problem

“An edge is critical if removing it disconnects the graph” $\Rightarrow$ for each edge, remove it, run BFS/DFS to see if the graph is still connected, then put the edge back.

- 1: **for** each edge $e \in \text{connections}$ **do**
- 2: $G' \leftarrow G$ with e removed
- 3: **if not** ISCONNECTED(G') **then**
- 4: add e to the answer
- 5: **end if**
- 6: **end for**

Why this is bad

- Each connectivity check costs $O(n + m)$.
- There are m edges to try $\Rightarrow O(m(n + m))$ total.
- Plug in the actual constraints: $n = m = 10^5 \Rightarrow$ roughly $10^5 \times 2 \times 10^5 = 2 \times 10^{10}$ operations. Even at 10^9 simple operations/second this is ~ 20 seconds of raw work – and Python is far slower than that per operation. This **will** time out.

The smell to notice (again)

Same pattern as Problem 1: “remove one thing, recheck everything from scratch, repeat for every thing.” That is always a hint that most of the recomputation is unnecessary – the question is *what invariant* lets you reuse work.

Structural insight #1: only tree edges can be bridges

- Run one DFS from any start node; every edge is either a **tree edge** (first time you reach that node) or a **back edge** (an edge to an already-visited ancestor – this week's cycle-detection idea, extended to undirected graphs).
- A back edge, by definition, sits on a cycle (the tree path down plus the back edge itself) $\Rightarrow$ removing it **cannot** disconnect the graph. Back edges are *never* bridges.
- This alone prunes the m candidates down to at most $n - 1$ tree edges – but we still need an $O(1)$ -ish test *per tree edge*, not another full BFS/DFS each time.

Structural insight #2: discovery time and low-link

For each node u , computed during one single DFS:

- $tin[u]$: the time (a counter, incremented once per visit) when u is first discovered.
- $low[u]$: the **smallest** tin value reachable from u 's subtree using at most *one* back edge – i.e., how far back up the tree can this subtree “reach” without using the edge to its parent.

$$low[u] = \min \left(tin[u], \min_{(u,v) \text{ back edge}} tin[v], \min_{(u,c) \text{ tree edge}} low[c] \right)$$

- **Bridge test:** tree edge $(parent, child)$ is a bridge $\iff low[child] > tin[parent]$ – i.e., $child$'s entire subtree has **no** back edge escaping past $parent$.

DFS trace on the worked example

Adjacency: 0: {1, 2}, 1: {0, 2, 3}, 2: {1, 0}, 3: {1}. DFS from node 0, timer starts at 0.

step	event	<i>tin</i>	<i>low</i>	remark
1	discover 0	0	0	start
2	discover 1 (tree 0–1)	1	1	
3	discover 2 (tree 1–2)	2	2	
4	edge 2–0: 0 already visited, not parent	–	$low[2]=0$	back edge: $low[2] \leftarrow \min(2, tin[0]=0)$
5	return 2 $\rightarrow$ 1	–	$low[1]=0$	$low[2]=0 \not> tin[1]=1$: not a bridge
6	discover 3 (tree 1–3)	3	3	
7	return 3 $\rightarrow$ 1	–	$low[1]=0$	$low[3]=3 > tin[1]=1$: bridge!
8	return 1 $\rightarrow$ 0	–	$low[0]=0$	$low[1]=0 \not> tin[0]=0$: not a bridge

Result: only (1, 3) satisfies $low[child] > tin[parent]$ – matches the expected output $[[1, 3]]$.

Optimal algorithm: pseudocode

Run DFS($s, -1$) from any unvisited s (repeat for every unvisited vertex, in case the graph is disconnected):

```
1: function DFS( $u$ , parentEdge)
2:    $tin[u] \leftarrow low[u] \leftarrow timer$ ;  $timer \leftarrow timer + 1$ ; mark  $u$  visited
3:   for each edge  $(u, v)$  with id  $e$  incident to  $u$  do
4:     if  $e = parentEdge$  then continue                                ▷ never re-cross the entry edge
5:     else if  $v$  unvisited then
6:       DFS( $v$ ,  $e$ )
7:        $low[u] \leftarrow \min(low[u], low[v])$ 
8:       if  $low[v] > tin[u]$  then report  $(u, v)$  as a bridge
9:       end if
10:    else
11:       $low[u] \leftarrow \min(low[u], tin[v])$                                 ▷ back edge: relax
12:    end if
13:  end for
14: end function
```

$O(n + m)$ total – the same traversal bound as plain BFS/DFS this week, plus two $O(1)$ bookkeeping arrays.

Complexity: naive vs. optimal

Approach	Time	At $n = m = 10^5$
Remove edge + recheck connectivity	$O(m(n + m))$	$\sim 2 \times 10^{10}$ ops
Prune to tree edges, still recheck each	$O(n(n + m))$	$\sim 2 \times 10^{10}$ ops
Single DFS with tin/low	$O(n + m)$	$\sim 2 \times 10^5$ ops

Five orders of magnitude apart – the difference between “times out” and “instant,” purely from finding the right invariant ($low[child] > tin[parent]$) instead of brute-force rechecking.

Python implementation (part 1: building the graph)

```
1 def criticalConnections(n, connections):
2     graph = [[] for _ in range(n)]
3     for edge_id, (u, v) in enumerate(connections):
4         graph[u].append((v, edge_id))
5         graph[v].append((u, edge_id))      # store the edge's own id, not just "
6                                             parent"
7
8     tin = [-1] * n
9     low = [0] * n
10    timer = 0
11    bridges = []
```

Storing the **edge id** alongside each neighbor (not just the neighbor itself) is what lets us correctly skip “the edge we just came from” in the next slide.

Python implementation (part 2a: descend / relax)

```
1  for root in range(n):
2      if tin[root] != -1:
3          continue
4      stack = [[root, -1, 0]]          # [node, incoming_edge_id, neighbor_cursor]
5      tin[root] = low[root] = timer
6      timer += 1
7      while stack:
8          node, in_edge, idx = stack[-1]
9          if idx < len(graph[node]):
10             stack[-1][2] += 1          # advance cursor before using it
11             nxt, eid = graph[node][idx]
12             if eid == in_edge:
13                 continue              # don't re-cross the entry edge
14             if tin[nxt] == -1:          # tree edge: descend
15                 tin[nxt] = low[nxt] = timer
16                 timer += 1
17                 stack.append([nxt, eid, 0])
18             else:                      # back edge: relax
19                 low[node] = min(low[node], tin[nxt])
```

Python implementation (part 2b: return / record bridges)

```
1         else:
2             stack.pop()                # subtree of node fully
3             explored
4             if stack:
5                 parent = stack[-1][0]
6                 low[parent] = min(low[parent], low[node])
7                 if low[node] > tin[parent]:
8                     bridges.append(connections[in_edge])
9
10    return bridges
```

This else branch fires once a node's neighbor cursor reaches the end of its list – the iterative equivalent of a recursive call *returning*: relax the parent's low and apply the bridge test right there.

Implementation tips & tricks (the programming side)

- **Recursive DFS will crash on this problem, not just run slowly.** A textbook-clean recursive version of the pseudocode above hits Python's default recursion limit (≈ 1000) whenever the graph contains a path of > 1000 nodes – entirely possible at $n = 10^5$. Raising `sys.setrecursionlimit` alone can still segfault (the C stack, not just Python's counter, overflows); the robust fix is either running the recursive version on a thread with a manually enlarged `threading.stack_size()`, or – as above – converting the recursion into an **explicit stack** that stores a resumable cursor into each node's neighbor list.
- **Track the incoming edge's id, not the parent node.** If a graph ever has multiple edges between the same pair (not this problem, but common in the “directed” sequel *Redundant Connection II*), “skip any edge back to the parent” silently discards a legitimate second edge. Skipping by exact `edge_id` is correct in general; skipping by parent *node* only happens to be correct here because the input has no duplicate edges.

Implementation tips & tricks (continued)

- **Preallocate arrays, don't grow dictionaries.** `tin = [-1] * n` and `low = [0] * n` are $O(1)$ -indexed from the start; a dict that grows one key at a time carries hashing overhead that is measurable at $n = 10^5$ in Python.
- `graph = [[] for _ in range(n)]`, **not** `defaultdict(list)`. Both work, but a plain list avoids `defaultdict`'s per-access dispatch inside the hottest loop in the program (every edge is touched twice, once per endpoint).
- **A one-element mutable “cursor” per stack frame is what makes the stack resumable.** The trick that makes the iterative version behave exactly like recursion-with-a-return-address is storing *where you left off* (`idx`) in each frame, so popping back up resumes the parent's loop instead of restarting it.

Summary: two problems, one lesson

- 1 **Redundant Connection** finds the one edge that *is* on the graph's (unique) cycle – solved by Union-Find, because the question is “are these two nodes already connected *right now?*”, asked incrementally as edges arrive.
- 2 **Critical Connections** finds every edge that is *on no cycle at all* – solved by a single DFS with *tin/low*, because the question is a *static* property of the whole graph, best answered by one traversal that remembers what it has already seen.
- 3 Both times, the naive instinct was “recompute connectivity from scratch for every candidate”; both times, the fix was the same shape: **find the invariant that makes one pass enough** – exactly the move this week's lecture made going from “recheck the whole tree” to $O(m \alpha(n))$ Union-Find.
- 4 The implementation tricks (iterative find/DFS, edge-id tracking, preallocated arrays) are a second, independent layer: even the *optimal algorithm* can fail in practice if the code hits language-level limits like recursion depth.

Keep practicing

LEETCODE.md lists eight more Week 3 problems (*Number of Islands*, *Course Schedule*, *Graph Valid Tree*, *Accounts Merge*, ...) – try applying the same four-step process to each of them.